Node.js — Day 7 Complete Notes

Topic: File System (fs) — Reading and Writing Files

Goal: Learn how Node.js works with files using the built-in fs module and connect asynchronous file operations to the Event Loop.

1. What is fs?

fs stands for File System. It is a built-in Node.js module used to work with files and directories.

```
const fs = require("fs");
```

Because fs is built into Node.js, it does not need to be installed with npm.

2. Reading a File with fs.readFile()

fs.readFile() reads the contents of a file asynchronously.

```
const fs = require("fs");

fs.readFile("./message.txt", "utf8", (err, data) => {
  if (err) {
    console.log(err);
    return;
  }

  console.log(data);
});
```

If message.txt contains Hello Bhavana!, the callback receives that text in data.

3. The readFile Callback

```
(err, data) => {
  // ...
}
```

err contains information about an error if the operation fails. data contains the file contents when the read succeeds. Checking err before using data is a common basic error-handling pattern.

4. What is utf8?

For our current level, remember that utf8 tells Node.js to return the file contents as readable text. Without an encoding, Node.js can return the contents as a Buffer; Buffers will be covered later when useful.

5. readFile() is Asynchronous

```
console.log("Start");

fs.readFile("./message.txt", "utf8", (err, data) => {
  console.log("File:", data);
});

console.log("End");
```

Expected order:

```
Start
End
File: Hello!
```

The file operation starts, but JavaScript continues with synchronous code. When the read is ready, the callback is processed. This is a practical example of asynchronous I/O and connects to the Event Loop concepts from Days 2–4.

6. Writing a File with fs.writeFile()

fs.writeFile() writes data to a file asynchronously.

```
const fs = require("fs");

fs.writeFile("./message.txt", "Hello Bhavana!", (err) => {
  if (err) {
    console.log(err);
    return;
  }

  console.log("File written successfully");
});
```

If the file does not exist, writeFile() can create it. If it already exists, writeFile() replaces its existing contents with the supplied content.

7. readFile vs writeFile

```
fs.readFile(file, encoding, callback);
// Reads file contents

fs.writeFile(file, data, callback);
// Writes/replaces file contents
```

8. Practical Mental Model

```
readFile:
File → read → data → callback

writeFile:
Data → write → file → callback
```

Both methods are asynchronous versions and fit into Node.js's asynchronous I/O model.

9. Important Rules

• fs means File System.

• fs is a built-in Node.js module.

• fs.readFile() reads a file asynchronously.

• fs.writeFile() writes/replaces file contents asynchronously.

• utf8 gives readable text for a file read.

• A read callback commonly receives (err, data).

• Check the error before using the result.

• Asynchronous file operations allow JavaScript to continue rather than synchronously waiting for the file result.

• writeFile() can create a file if it does not already exist.

10. Day 7 Quick Revision

fs: Built-in File System module.

readFile(): Read file contents.

writeFile(): Write/replace file contents.

utf8: Return file contents as readable text.

err: Error information if the operation fails.

data: File contents returned by a successful read.

Async I/O: File work can be handled without making JavaScript synchronously wait for the result.

Key Takeaway

Node.js's fs module lets backend applications work with files. readFile() and writeFile() are asynchronous operations and are practical examples of asynchronous I/O and the Event Loop concepts learned earlier.